STAMPMYVISA · AI OPS ENGINEER · HIRING ASSIGNMENT · CONFIDENTIAL

The Build Log

Rithesh Suvarna — AI Ops Engineer hiring assignment (StampMyVisa). Tool used: **Claude Code** (Anthropic's CLI agent), Sonnet model, working directly in this repo's working directory for the entire session — no other AI tool was used.

Submission 3 Real prompts quoted verbatim, not cleaned up

This is a narrated log of the actual session: real prompts (quoted verbatim), what the AI actually did in response, what broke, what it found on its own and flagged, and how each pivot happened. Nothing here is reconstructed after the fact — it's written from the session's own history. Tool-call-level noise (every individual file read/write) is summarized rather than dumped verbatim, since the raw tool trace runs to hundreds of calls; what's preserved is every decision point, every bug, every redirect.

PHASE 0

Analysis before code

~08:00–08:15

PROMPT

"Analyze the Relocation Project. I want to build a production-ready AI-powered Relocation Operations Tracker for the QuickMove Hiring Assignment. Do NOT start coding. First analyze everything. Read the complete Hiring Assignment document… [business requirements, ops workflow, stakeholders, existing manual process, customer journey, dependencies, city variations, edge cases, failure modes, automation opportunities, AI opportunities, success metrics]… Now analyze the existing relocation tracker reference (if provided)… Break down the application into modules… Design the complete application architecture… List every feature required… Identify all possible edge cases… Create a complete implementation roadmap… Do NOT write code. Wait for my approval before implementation." *(abbreviated — the full instruction ran to ~40 lines)*

What I did: Found two files already sitting in the Downloads folder — `assignment_ai_ops_engineer.pdf` (the actual hiring brief) and `quickmove_tracker.jsx` (an existing single-file React prototype, apparently from an earlier session). Read both in full. Entered **plan mode** explicitly, since this matched the tool's own criteria (architectural decisions, multi-file scope, explicit "wait for approval" instruction).

Analysis produced: business/ops breakdown (15+ workflows, stakeholders, city variation, manual-process failure modes), a critique of the reference prototype (found a real security bug in it — it called the Anthropic API directly from the browser with no key, which would either fail or leak a key if "fixed" naively; also flagged: no backend, no per-city templates, no audit trail, no responsive layout), a ranked top-3 automation opportunity list, and a target production architecture (Next.js, Postgres via Prisma, server-side AI calls). Wrote this to a plan file and called `ExitPlanMode`.

Outcome: approved by the user via the plan-mode UI.

PHASE 1

Building the app, then a full spec change

~08:15–09:00

Before I could act on the approved plan, the user's next message arrived mid-turn with a **completely different tech stack** than what I'd just gotten approved: Vite (not Next.js) + separate Express backend + Zustand + React Query + Framer Motion + React Hook Form + React Router, Railway/Vercel for deploy, "Continue improving until the application is production-ready… Do not stop after partial implementation."

Decision: I didn't push back on the stack change or re-litigate the plan — the user had made an explicit, detailed call, so I adapted the plan's *content* (data model, escalation logic, AI-safety patterns) onto the *new* stack rather than defending my original Next.js recommendation. Built:

- **Backend:** Express + Prisma + SQLite (chosen over Postgres for zero-setup speed; schema written to be Postgres-portable), Zod validation, a Claude API service with retry-on-invalid-JSON, an escalation service that persists blocked/stalled state instead of just computing it client-side.
- **Frontend:** Vite + React + Tailwind v4 + the full requested stack, with an offline-first data layer — every API call falls back to `localStorage` transparently if the backend is unreachable.

Bugs found and fixed this phase, by testing in a real browser

1. **Move dates displayed a day early.** `toLocaleDateString` with no timezone pinned meant a UTC-midnight date shifted backward in any timezone behind UTC. Fixed by adding `timeZone: "UTC"` everywhere a move date is formatted (both frontend and the AI message-drafter prompt on the backend).
2. **Duplicate error message in the Add Relocation form.** The same-city validation error was rendering twice — once via the shared `FormField` component, once via a leftover manual paragraph I'd added redundantly. Removed the duplicate.
3. **The offline-fallback logic never actually triggered locally.** I killed the backend process to test the fallback path, and instead of the offline banner appearing, the app just showed an empty dashboard silently. Root cause, found by reading the actual network log: Vite's dev `/api` proxy turns a downed backend into an HTTP `502` response rather than a connection failure, so my `isNetworkError = !err.response` check saw a real response object and never fell back. Fixed by treating 502/503/504 *without* our own API's JSON error shape as "unreachable," while still letting our own legitimate 503s (e.g. "AI not configured") pass through as themselves — otherwise that fix would have broken the AI-not-configured message by misclassifying it as downtime.

Verified end-to-end in the browser: CRUD, checklist status changes with optimistic updates, escalation auto-detection, search/filter, mobile responsive layout (a genuine master-detail pattern, not just a squeezed desktop view), duplicate/same-city validation, and the offline fallback (confirmed by actually killing the backend process and watching `localStorage` populate).

PHASE 2

Git, GitHub, and "give me the entire project folder"

> PROMPT
>
> `"i need to send the link to submit the project and push to github so give me the entire project folder"`

I checked the environment first rather than assuming: no `gh` CLI, no `vercel` / `railway` CLI installed, git not yet initialized. Rather than guess at the user's GitHub/deployment setup, I asked two direct questions (repo situation, deployment platform preference) instead of just picking for them, since those are genuinely their calls, not mine.

Their answers: "just give me the exact commands" for GitHub (they'd push themselves), "walk me through Vercel + Railway dashboards" for deploy.

I `git init`'d, wrote a root `.gitignore`, caught and excluded a `.claude/settings.local.json` file that `git add -A` had swept up (local tool-permission history, not project source — correctly excluded, not a secret but not something that belongs in the repo either), committed, and built a clean zip via `git archive` (guarantees no `node_modules` / `.env` leak into it, since it only includes tracked files) as a submittable fallback independent of GitHub/deploy working out.

Gave exact `git remote add` / `push` commands rather than pushing myself, respecting their stated preference.

PHASE 3

"cant i do in one?"

The user pushed back on the two-platform deploy (Vercel + Railway) I'd just walked them through. Rather than defend the original design, I found a real simplification: **have Express serve the built frontend directly**, so the whole app is one service, one URL, no CORS, no cross-origin env var.

Implementation: `backend/src/app.js` now checks if `frontend/dist` exists and, if so, serves it as static files with an SPA fallback (`app.get("*", …)` → `index.html`) — scoped so it never interferes with local dev, where the frontend runs its own Vite server and `dist` doesn't exist yet. Tested by actually building the frontend and hitting the backend's root URL directly, confirming the SPA route (`/relocations/:id`) correctly falls back to `index.html` instead of 404ing, and that `/api/*` 404s stayed real 404s instead of getting swallowed by the SPA catch-all.

PHASE 4

The Railway deploy loop

the messiest part — worth the detail rather than summarizing it away

Attempt 1 — `"railpack process exited with an error."` No further detail in that first message. My hypothesis, before seeing any real log: Railway's Railpack build system needs a `package.json` at whatever directory is set as the build root to even recognize the project as Node.js — and since I'd pointed Root Directory at the repo root (for the single-service setup) and there was no root-level `package.json` (only `backend/` and `frontend/` had one), Railpack likely failed before reaching any of my custom build/start commands. I added a root `package.json` with orchestration scripts, and tested both scripts locally end-to-end before telling the user to redeploy — including deliberately stopping my local dev backend, running `npm start` from the repo root exactly as Railway would, and curling it to confirm.

Attempt 2 — user pasted a real build log showing `node | 18.20.8 | package.json > engines > node (>=18)` and the build still failing. I noticed nearly every build step said `cached` — the giveaway that Railway had reused a stale snapshot from *before* the fix landed, most likely because clicking "Redeploy" on an old failed deployment re-runs that exact historical snapshot rather than pulling the latest commit. I verified independently (not just taking the user's word) by running `git fetch origin` and diffing — confirmed the fix commit really was on `origin/main`, so the problem was on Railway's side, not a failed push. Told the user specifically not to use "Redeploy" on the old failure and to trigger a fresh deploy instead.

Attempt 3 — user pasted a build log that actually succeeded (`npm install`, `npm run build`, image pushed) but then a **runtime crash loop**: `Error: Environment variable not found: DATABASE_URL.` Also noticed the service name had silently changed from `stampmyvisaaa` to `clever-youth` — a strong signal this was a fresh service that never got the env vars I'd had them set on the old one. Told them exactly what to add and where.

Attempt 4 — build failed again, this time on `npm run build` itself, log showing `Node.js v18.20.8` printed right above the failure with no further detail — I recognized this specific pattern (a bare Node version line right before a crash) as Node's own uncaught-exception banner, not application output, meaning something threw before it could print a real error. I checked my *own* local Vite install's `engines` field rather than guess: `"node": "^20.19.0 || >=22.12.0"`. My root `package.json`'s `"node": ">=18"` was too loose — Railpack had picked the oldest version satisfying that range (18.20.8), which Vite 8 doesn't support at all. This had never surfaced locally because my own dev machine is on Node 24. Fixed the `engines` field to `>=20.19.0` in all three `package.json` files, rebuilt locally to confirm nothing else broke on the new constraint, committed with a message that explains *why*, so the reasoning survives in git history, not just the diff.

Deploy succeeded. User: "its online now."

PHASE 5

Finding the URL, and a real smoke test

The user asked "where do i get the url" — I'd noticed the earlier screenshot said "Unexposed service," which specifically means no public domain exists yet, not that one is hidden somewhere. Walked them through Generate Domain. They asked what port to enter; I answered 4000 (matching the project's documented default) but then they picked 8080 instead — I confirmed that's fine rather than making them redo it, since the app already reads `process.env.PORT` dynamically and Railway injects whatever port you tell it to route to.

Once they gave me the live URL, I ran a **real smoke test** rather than taking "its online" at face value:

- Bash's `curl` against the public URL failed outright (exit code 43 / connection refused) — this sandbox's shell apparently has no outbound internet access, only localhost. Switched to the browser tool instead, which worked.
- Created a real relocation, flipped a checklist item to Done, confirmed it persisted via a page reload and via the network log showing real `200` / `201` responses against the live domain (not the offline fallback).
- Tested the AI parser against production: **found a bug in my own test script**, not the app — I'd grabbed the native value-setter for `HTMLInputElement` and used it on a `<textarea>` (which needs `HTMLTextAreaElement`'s setter), so the field silently stayed empty and no request fired. Caught this by checking the network log and seeing no `/api/ai/*` request at all, diagnosed it correctly, fixed the test script, reran it, and confirmed the real behavior: a genuine `503` from the live server with the correct "AI not configured" message, since no `ANTHROPIC_API_KEY` is set on that service.
- Checked mobile responsiveness on the live deploy the same way I had locally, confirmed via `getComputedStyle` (not just visual guessing) that the sidebar-only mobile layout was actually applying (`display: none` on `main` at 375px width), since the page-text extraction tool doesn't respect CSS visibility and would have given a false read.
- Cleaned up the test relocation afterward so the link doesn't show stale demo data to whoever opens it next.

Also talked through the tradeoff of skipping a persistent volume (SQLite resets on every redeploy without one) — gave the honest answer that it's fine for a demo link reviewed once or twice, not fine for anything longer-lived, and let the user make that call rather than deciding for them.

PHASE 6

The Map (Submission 1)

Went back to the business analysis from Phase 0 and turned it into an actual document (`docs/the-map.md`): a mermaid dependency diagram, a stakeholder table, a 31-workflow inventory (caught and fixed my own arithmetic error here — an earlier draft said "28+" from before I'd finished adding items; recounted and corrected to the actual number, 31, in both the intro line and the cross-reference in the ranking section), the same ranked top-3 automation opportunities from the plan, per-workflow health metrics, and the "hidden" edge cases section.

I asked the user what format they wanted (markdown / visual artifact / both) — they didn't answer that specific question, so rather than block on it I made a call: build the markdown draft first since it's the most editable and lowest-risk, and left the visual-artifact option open for later rather than assuming.

When they later said "you do it" and gave their name: filled in the name, then designed and built a visual HTML artifact. Deliberately avoided the most common AI-generated design cliché — I checked my first instinct (warm cream background + serif display + terracotta accent) against the design skill's own explicit list of overused looks, recognized it matched almost exactly, and threw it out — in favor of a palette actually grounded in the subject: a civic-registry/transit-manifest aesthetic (slate-sage paper, teal/mustard/brick semantic colors, a slab-serif for headers, monospace for data labels), fitting a domain genuinely full of government paperwork and multi-city logistics.

> **Verification gap, disclosed honestly:** tried to visually confirm the published artifact renders correctly using the browser tool, but hit two sandbox limits — no claude.ai auth in that browser pane, and a policy block on local preview ports. Said so directly rather than claim a visual check I hadn't actually done, and asked the user to look at the real link themselves.

PHASE 7

"submission 1,2,3"

Terse status-check prompt. Rather than guess what they wanted, gave a straight status table across all three submissions — and in doing so, caught that Submission 3 itself didn't exist yet as a document, only as this session's raw history. Compiled it immediately: this document, written directly from the session's own history, preserving real quoted prompts and being explicit about the mistakes made along the way rather than editing them out.

PHASE 8

"yes covert to pdf"

Asked to convert the Map to PDF. Checked what tools actually existed in this environment rather than assuming a PDF library would just work: found Chrome and Edge both installed, and recognized a proper HTML→PDF conversion (preserving the actual CSS design) needed a real browser's print engine, not a from-scratch PDF-generation library.

Caught a rendering gap before it became a bug in the deliverable: the Map artifact's dependency diagram only renders as an actual diagram inside the Artifact platform's own hosted viewer — the raw static HTML file has no diagram library bundled into it. Printing the raw file directly would have produced a PDF with unrendered diagram *source code* sitting where the diagram should be. Caught this by reasoning through how the artifact actually gets rendered, before running the print command.

Fix: built a print-specific HTML variant with the same content and visual language, with the diagram replaced by a pure-CSS dependency-chain diagram — same information, renders reliably in any engine, no JS dependency. Rendered via headless Chrome. **Verified rather than assumed it worked:** extracted the text back out and checked for the name, the workflow count, and key phrases; separately rendered the PDF pages to images and read them back to visually confirm the colors, diagram, and layout had actually come through — not just that a file of nonzero size existed.

> **This document, following the same process:** the print variant you're reading now was built the same way, for the same reason.

REFLECTION

Decisions made without asking, and why

- **SQLite over Postgres** for the database — zero external signup, portable schema (documented the exact provider-swap steps in the README for later). Asked the user once via a structured question; they said "no preference," so I went with my stated recommendation rather than re-asking.
- **JavaScript over TypeScript** for both frontend and backend — the assignment's own rules hedge on this, and the scope was already large; optimized for shipping a complete, tested feature set over type coverage.
- **Server-side-only AI calls** — the original reference prototype called the Anthropic API directly from the browser, a real security bug. Non-negotiable fix, not something I flagged as optional.
- **Never auto-applying AI suggestions** — both AI features return suggestions for a human to review and approve; carried this pattern over from the reference prototype because it was the one part of it that was already correct.

What I'd flag as unfinished if this were real production work

Documented in the README's "Known limitations" section rather than hidden: minimal auth (name-selection, no passwords — fine for 5 known internal users), no data-sync strategy for a device that goes offline and comes back online later, no automated test suite (verification was a real manual pass through the golden path and edge cases, not unit tests).

Start time: ~08:03 AM PDT · End time: [fill in once actually sent] · 2026-07-24. Reconstructed from file-system timestamps (project folder creation, first plan-file write) since no direct session-start record exists.